

머신러닝과 데이터 처리

01 데이터 탐색

목차 01 데이터 탐색

1. 탐색적 데이터 분석
2. 데이터 처리를 위한 Pandas
3. 시각화를 위한 Matplotlib

1. 탐색적 데이터 분석



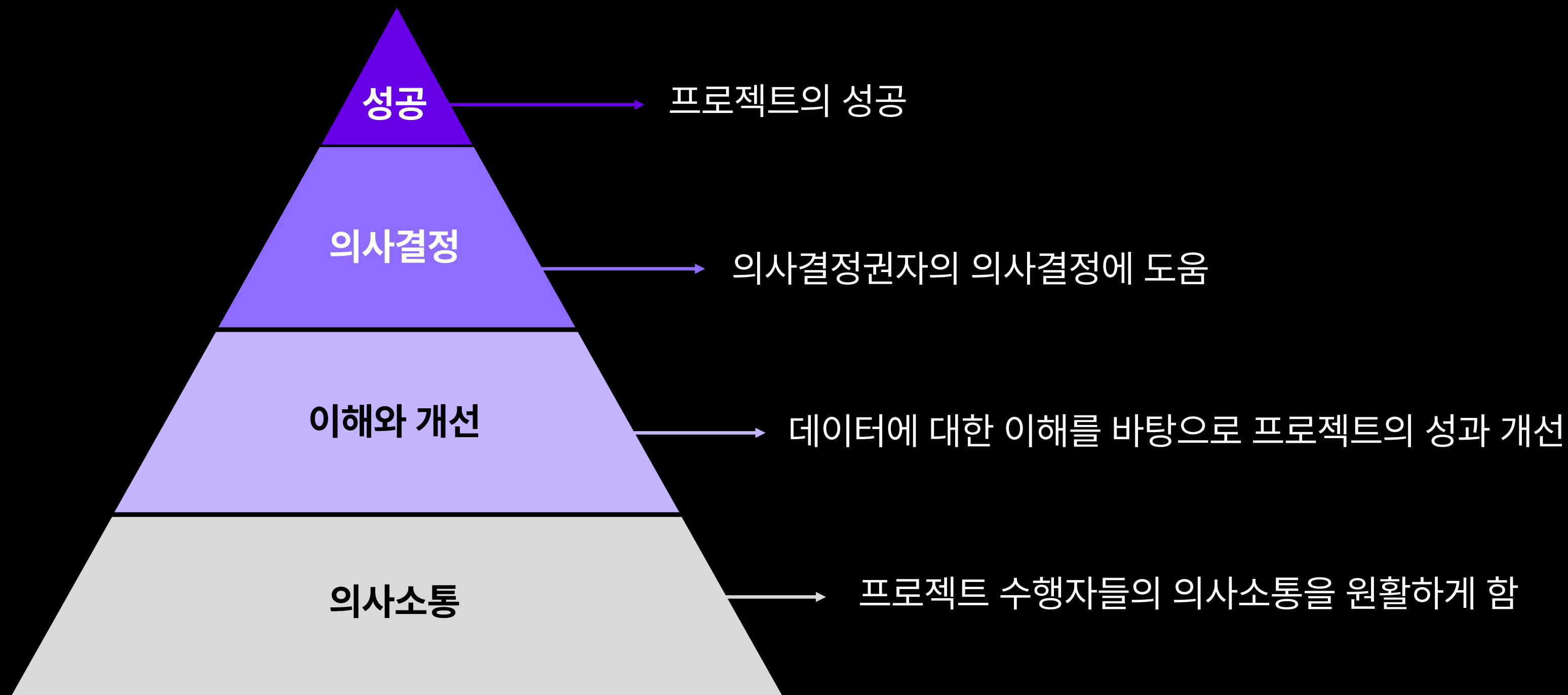
- 1977년, 미국 벨 연구소의 존 튜키 박사에 의해 제안된 개념
 - EDA(Exploratory Data Analysis)
- 가설 검증이나 모형 적용 전 데이터가 가진 정보를 사람에게 전달하는 방법
- 적절한 모형, 시각화 산출물을 사용하여 다양한 방식으로 데이터를 표현하고 상관관계를 분석

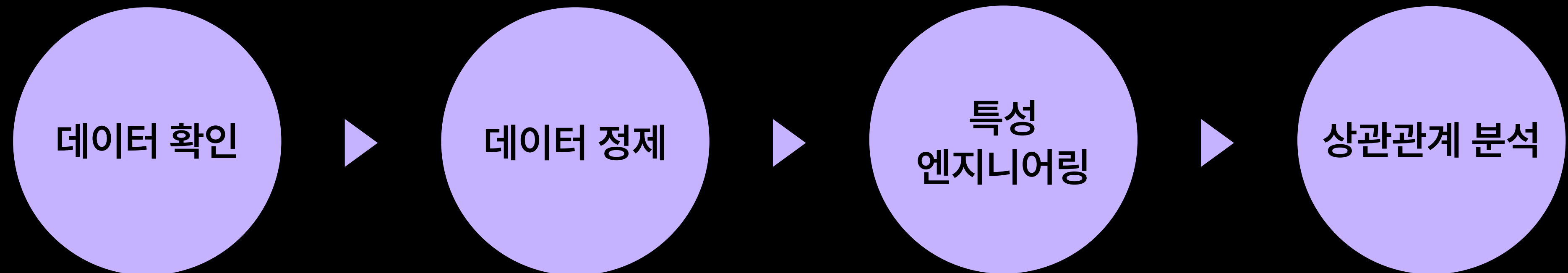


식재료



음식





탐색적 데이터 분석의 목적을 달성할 때 까지 각 단계를 순차적으로 반복



탐색적 데이터 분석을 수행하기 위해

머신러닝과 데이터 처리

데이터 탐색

탐색적 데이터 분석

	PassengerId	Survived	Pclass
0	1	0	3
1	2	1	1
2	3	1	3

데이터



데이터 조작 도구



시각화 도구

2. 데이터 처리를 위한 Pandas

- 파이썬 라이브러리



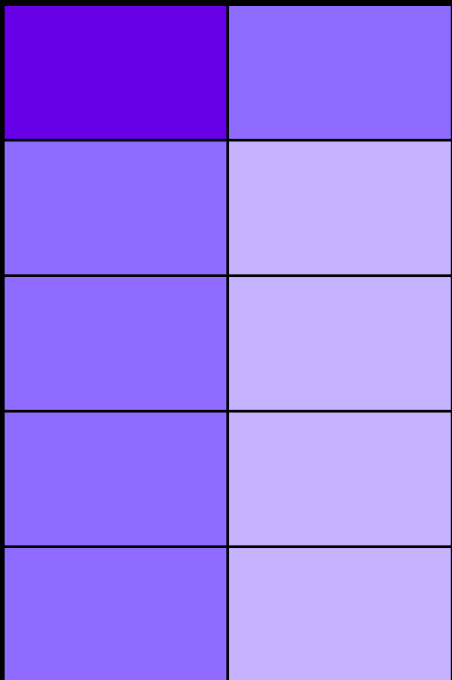
- 구조화된 데이터를 효과적으로 처리하고 저장

- NumPy를 기반으로 설계, 빠른 속도



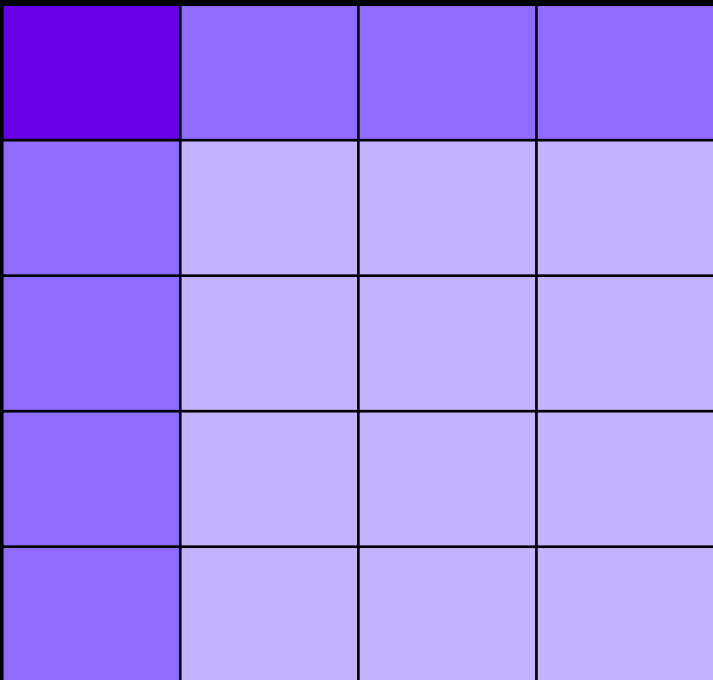
1차원

Series



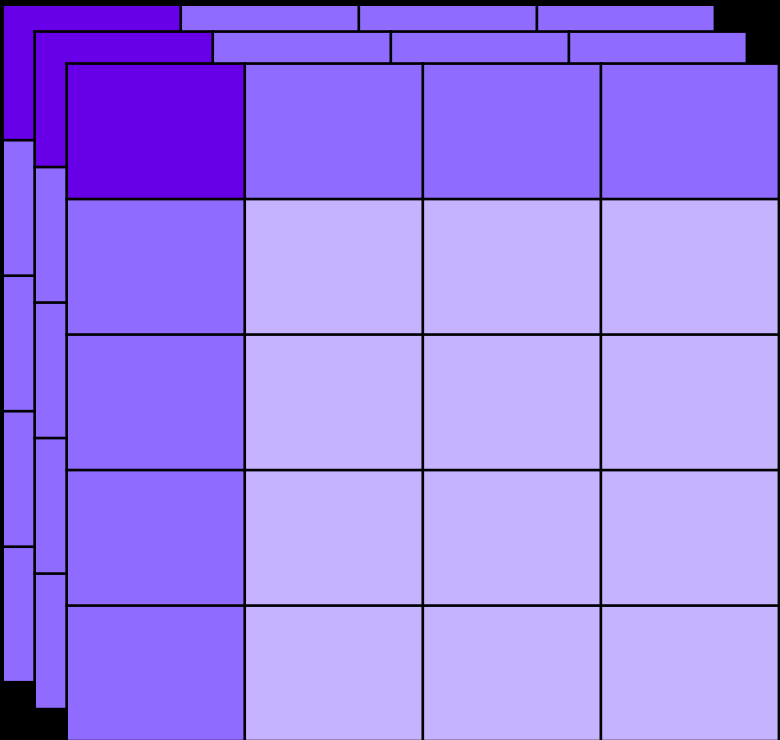
2차원

DataFrame



3차원

Panel





1차원 데이터 (Pandas Series)

머신러닝과 데이터 처리

데이터 탐색

데이터 처리를 위한
Pandas

서울의 월별 평균기온	평균기온
2022-01	-2.2
2022-02	-1.1
2022-03	7.7
2022-04	14.8

```
seoul_month_avgtemp = pd.Series([-2.2, -1.1, 7.7, 14.8],  
                                index=['2022-01', '2022-02', '2022-03', '2022-04'],  
                                name='avgTemp')
```



2차원 데이터 (Pandas DataFrame)

머신러닝과 데이터 처리

데이터 탐색

데이터 처리를 위한 Pandas

서울의 월별 기온	평균기온	최고기온	최저기온
2022-01	-2.2	8.4	-11.3
2022-02	-1.1	11.5	-10.7
2022-03	7.7	19.5	-3.0
2022-04	14.8	28.7	3.5

[illegible]



3차원 데이터 (Pandas Panel)

서울의 월별 기온				
		평균기온	최고기온	최저기온
2022-01		-2.2	8.4	-11.3
2022-02		-1.1	11.5	-10.7
2022-03		7.7	19.5	-3.0
2022-04		14.8	28.7	3.5

지역별 월별 기온		평균기온	최고기온	최저기온
서울	2022-01	-2.2	8.4	-11.3
	2022-02	-1.1	11.5	-10.7
	2022-03	7.7	19.5	-3.0
	2022-04	14.8	28.7	3.5
부산	2022-01	4	13.2	-6.1
	2022-02	3.6	15.2	-7.5

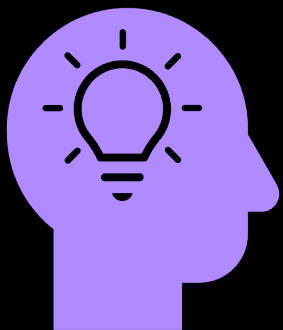
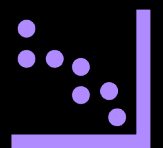
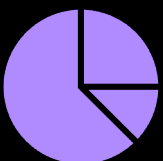
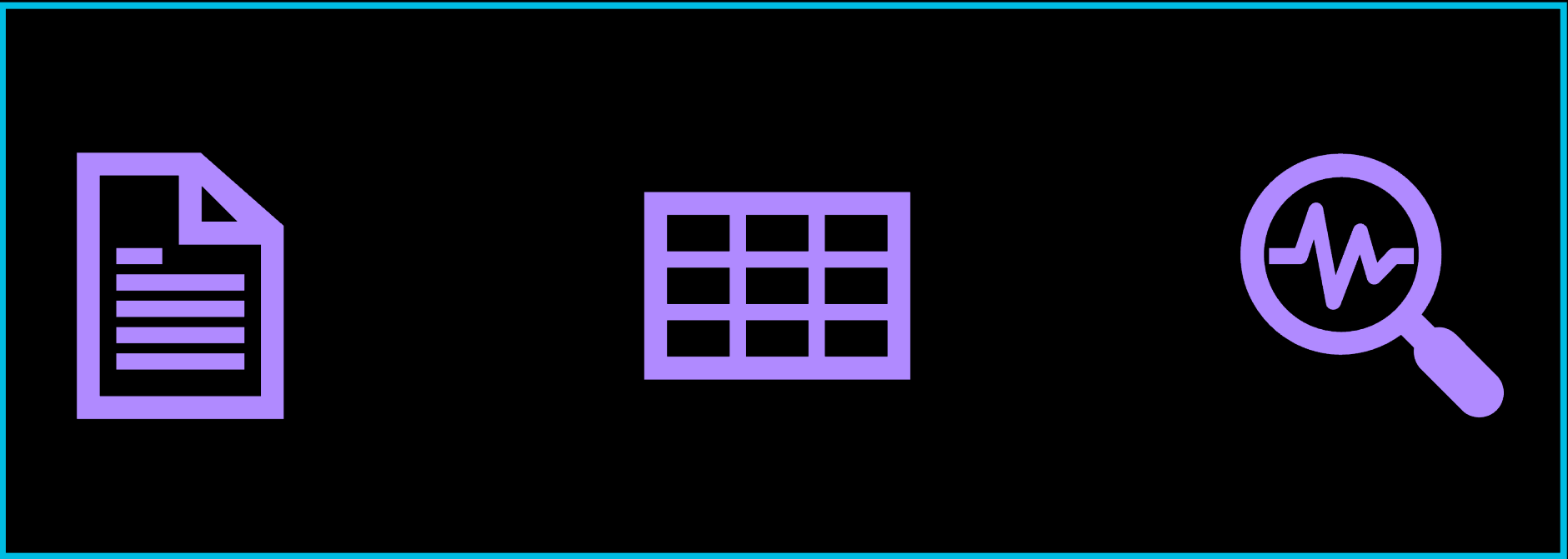


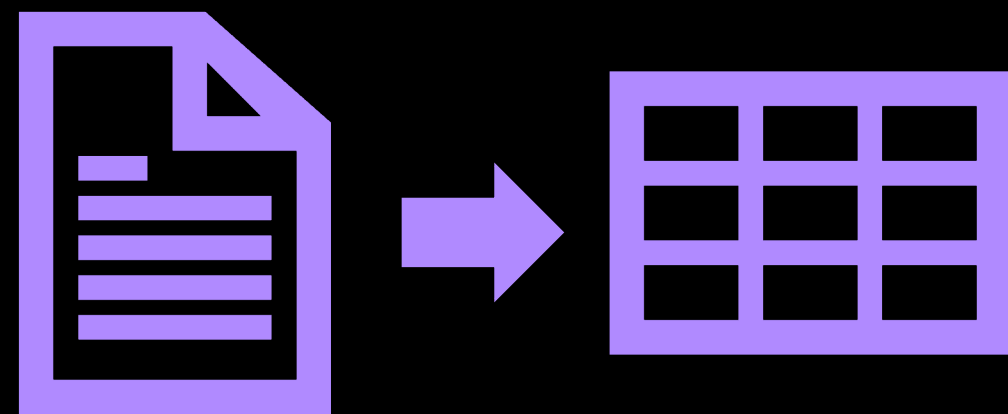
탐색적 데이터 분석과 Pandas

머신러닝과 데이터 처리

데이터 탐색

데이터 처리를 위한
Pandas





- Pandas는 csv, excel, json, html 등 거의 대부분의 데이터를 지원
- 데이터를 데이터프레임으로 만드는 데이터 불러오기는 read
(예 : read_csv(), read_excel())
- 데이터프레임을 데이터로 저장하기는 to
(예 : to_csv(), to_excel())



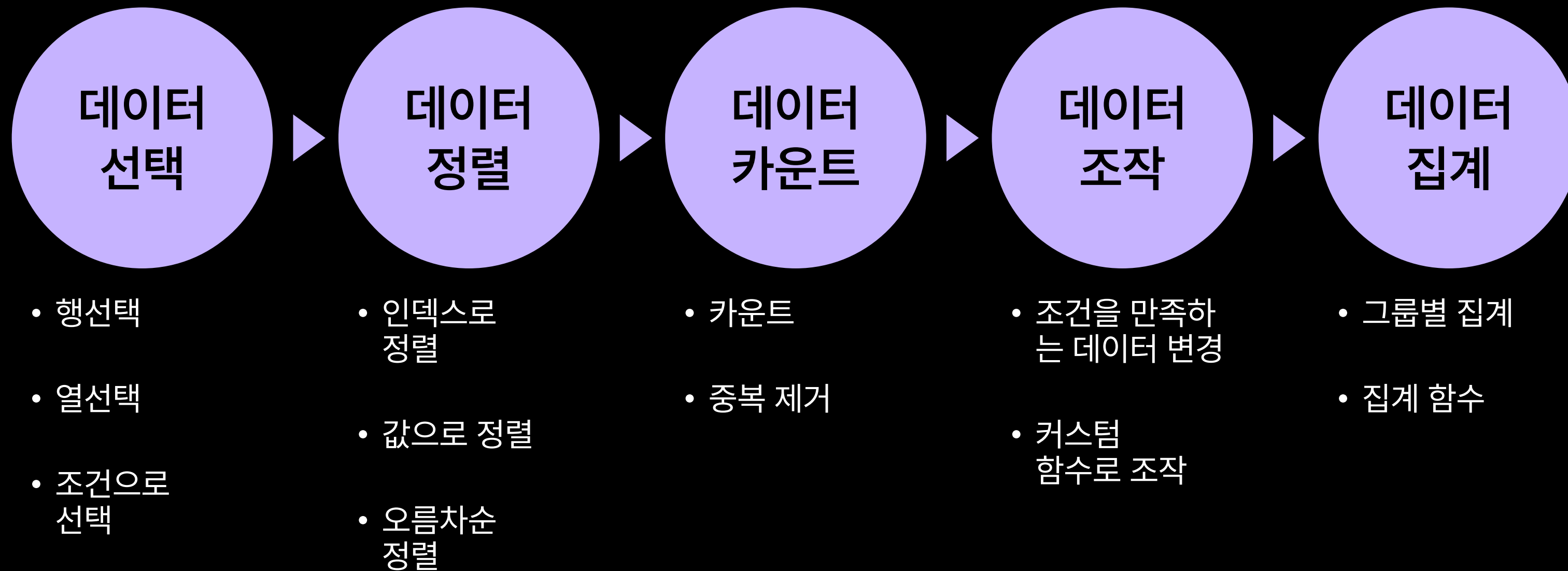
DataFrame 구조

서울의 월별 기온	평균기온	최고기온	최저기온
2022-01	-2.2	8.4	-11.3
2022-02	-1.1	11.5	-10.7
2022-03	7.7	19.5	-3.0
2022-04	14.8	28.7	3.5

Column

```
df['최저기온'].iloc['2022-04']
```

Index





행 선택

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
df.loc[0]  
df.iloc['2022-01']
```

열 선택

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
df['평균기온']
```

조건으로 선택

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
df[df['평균기온'] < -2]['평균기온']
```



인덱스로 정렬

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
df.sort_index()
```

값으로 정렬

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
df.sort_values(by=['평균기온'])
```

오름차순, 내림차순

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
df.sort_values(by=['평균기온'], ascending=True)
```



카운트

서울의 일별 날씨	날씨	평균기온
2022-02-01	맑음	8.4
2022-02-02	흐림	11.5

```
df["날씨"].value_counts()
```

중복제거

서울의 일별 날씨	날씨	평균기온
2022-02-01	맑음	8.4
2022-02-02	흐림	11.5

```
df["날씨"].unique()
```




조건을 만족하는 데이터 변경

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
df.loc[df['평균기온'] < 0, '평균기온'] = '영하'
```

커스텀 함수로 데이터 조작

서울의 월별 기온	평균기온	최고기온
2022-01	-2.2	8.4
2022-02	-1.1	11.5

```
def drop_outlier(num : int):  
    if num > 10 :  
        return 0  
    else :  
        return num  
  
df['최고기온'] = df['최고기온'].map(drop_outlier)
```



그룹별 집계

서울의 일별 날씨	날씨	최고기온
2022-02-01	맑음	8.4
2022-02-02	흐림	11.5

```
df.groupby(by="날씨")
```

집계 함수

서울의 일별 날씨	날씨	최고기온
2022-02-01	맑음	8.4
2022-02-02	흐림	11.5

```
df.groupby(by="날씨")["최고기온"].mean()  
df.groupby(by="날씨")["최고기온"].min()  
df.groupby(by="날씨")["최고기온"].max()  
df.groupby(by="날씨")["최고기온"].median()
```

3. 시각화를 위한 Matplotlib



- 파이썬 라이브러리

- 데이터를 그래프나 차트로 시각화할 수 있는 라이브러리



그래프 그려보기

머신러닝과 데이터 처리

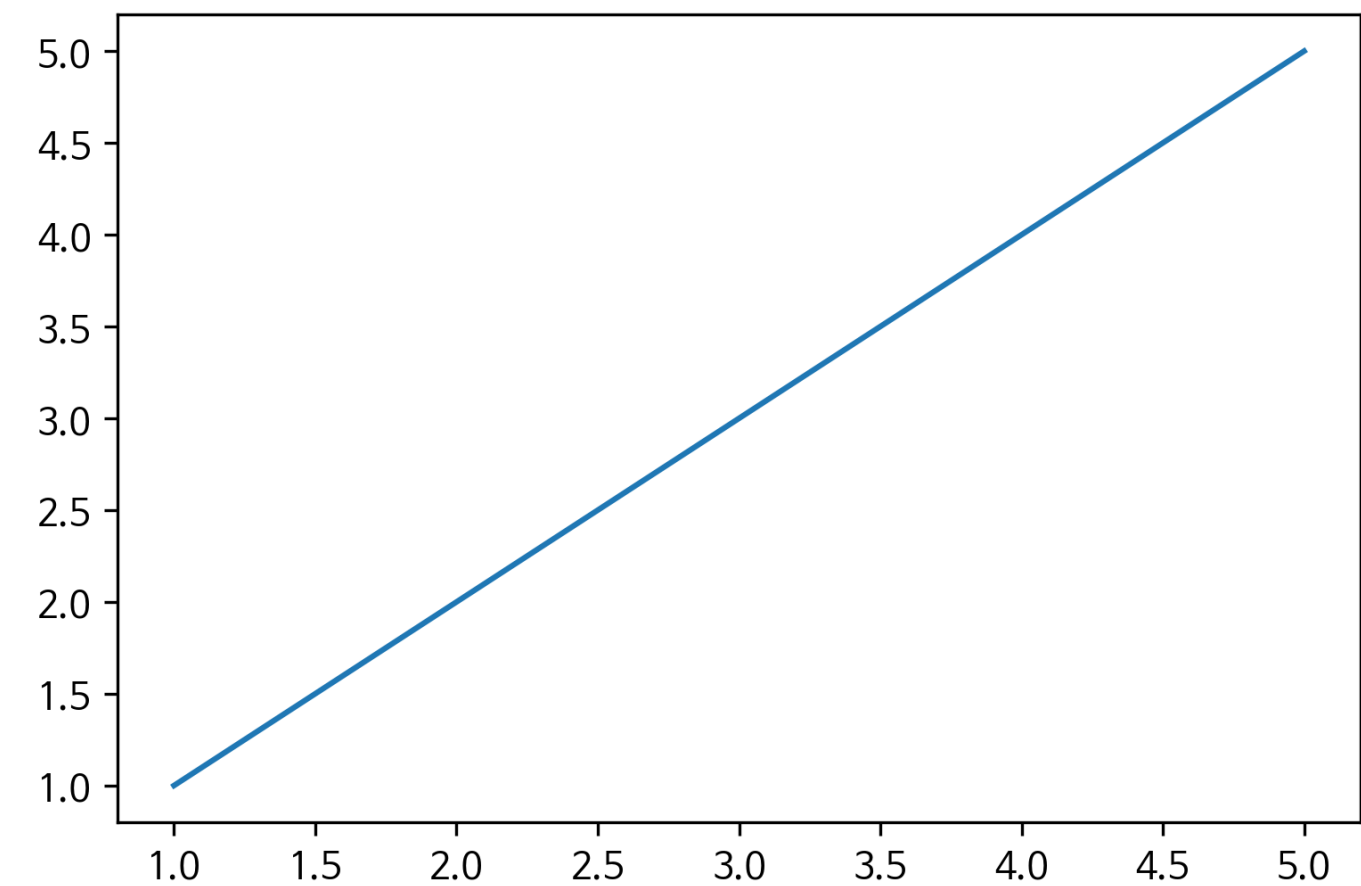
데이터 탐색

시각화를 위한
Matplotlib

```
x = [1, 2, 3, 4, 5]
```

```
y = [1, 2, 3, 4, 5]
```

```
plt.plot(x, y)
```





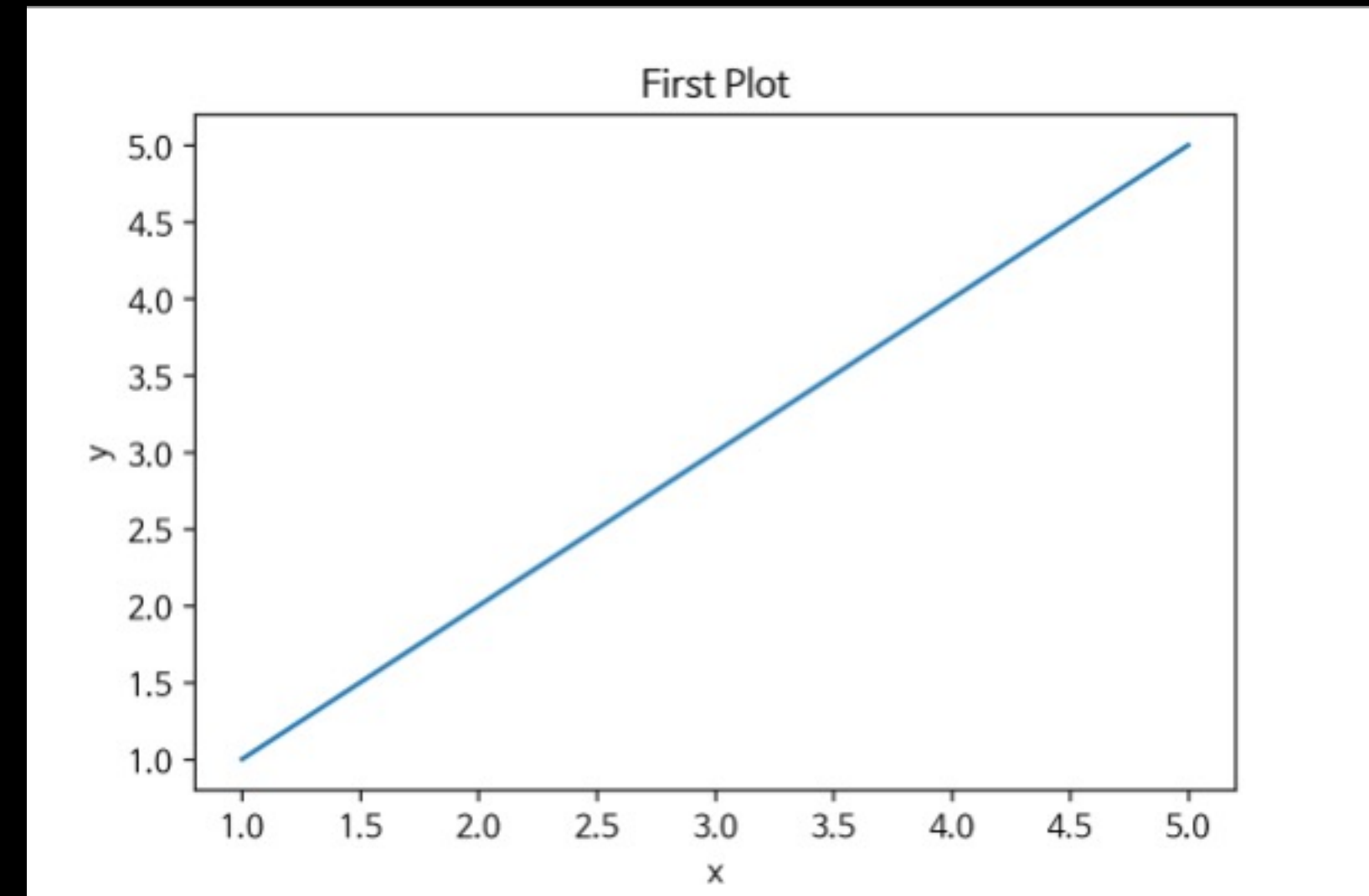
그래프 그려보기

머신러닝과 데이터 처리

데이터 탐색

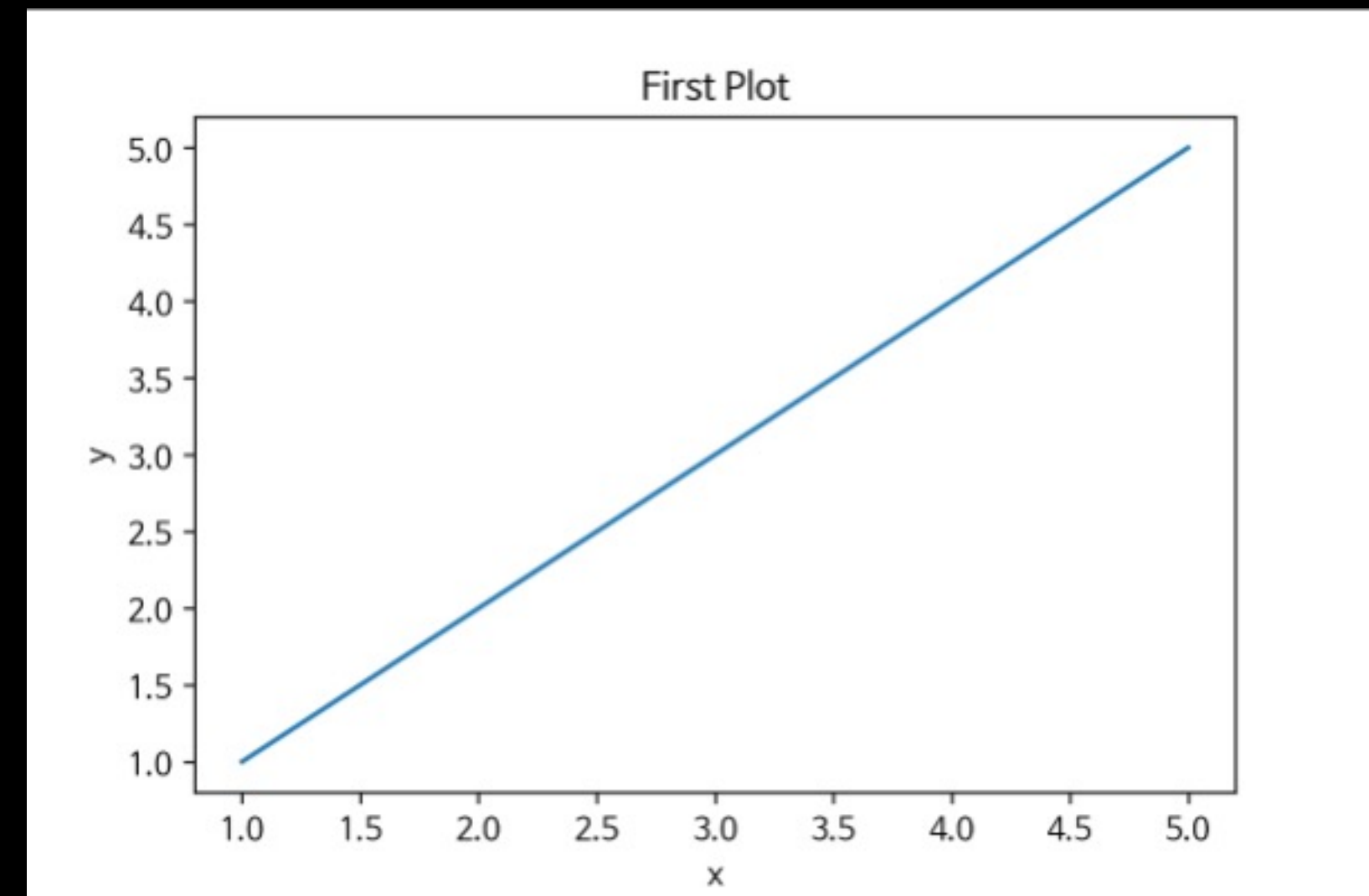
시각화를 위한
Matplotlib

```
x = [1, 2, 3, 4, 5]
y = [1, 2, 3, 4, 5]
plt.plot(x, y)
plt.title("First Plot")
plt.xlabel("x")
plt.ylabel("y")
```





```
x = [1, 2, 3, 4, 5]
y = [1, 2, 3, 4, 5]
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("First Plot")
ax.set_xlabel("x")
ax.set_ylabel("y")
```





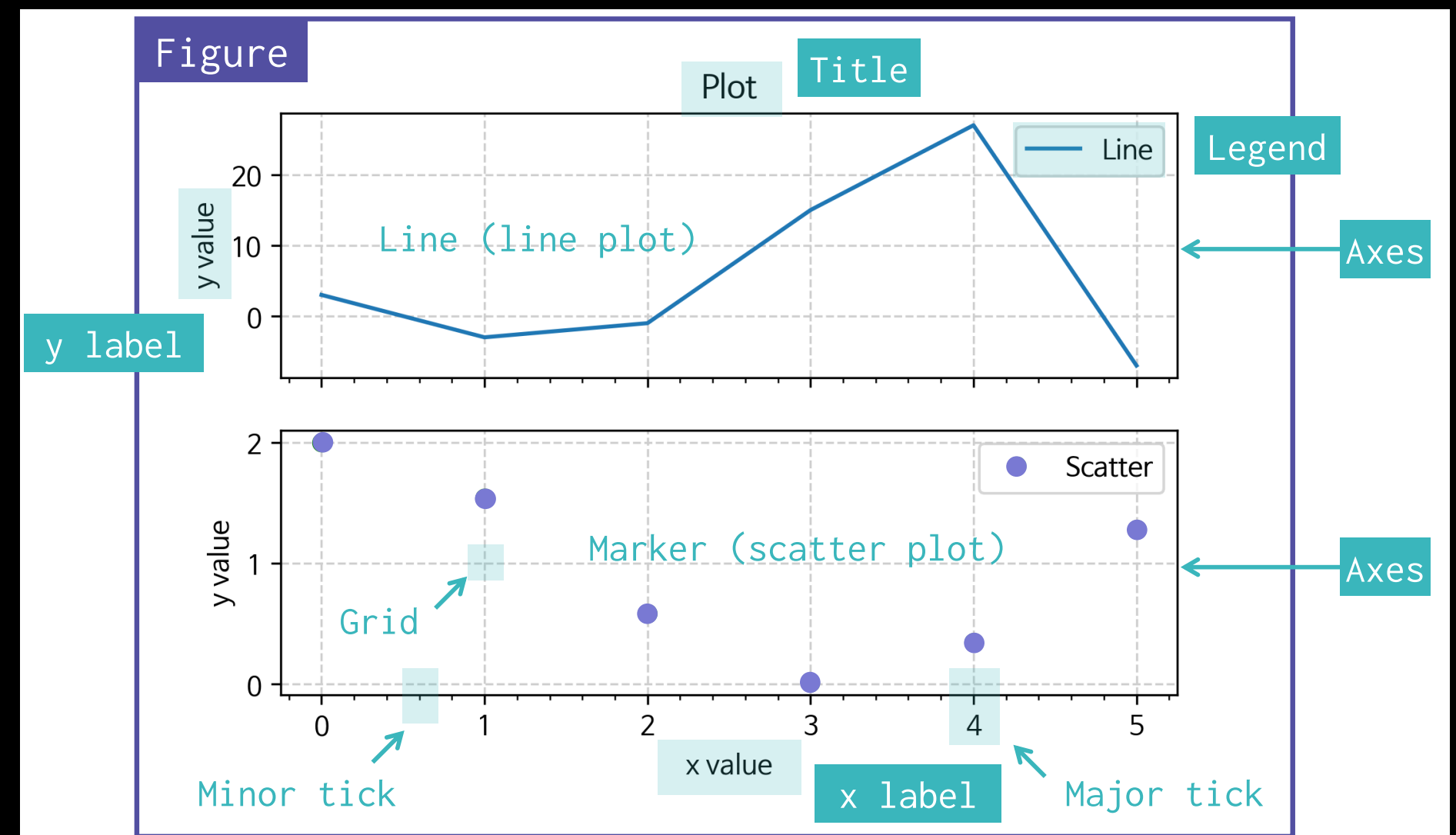
그래프 그려보기

머신러닝과 데이터 처리

데이터 탐색

시각화를 위한
Matplotlib

```
x = [1, 2, 3, 4, 5]
y = [1, 2, 3, 4, 5]
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("First Plot")
ax.set_xlabel("x")
ax.set_ylabel("y")
```





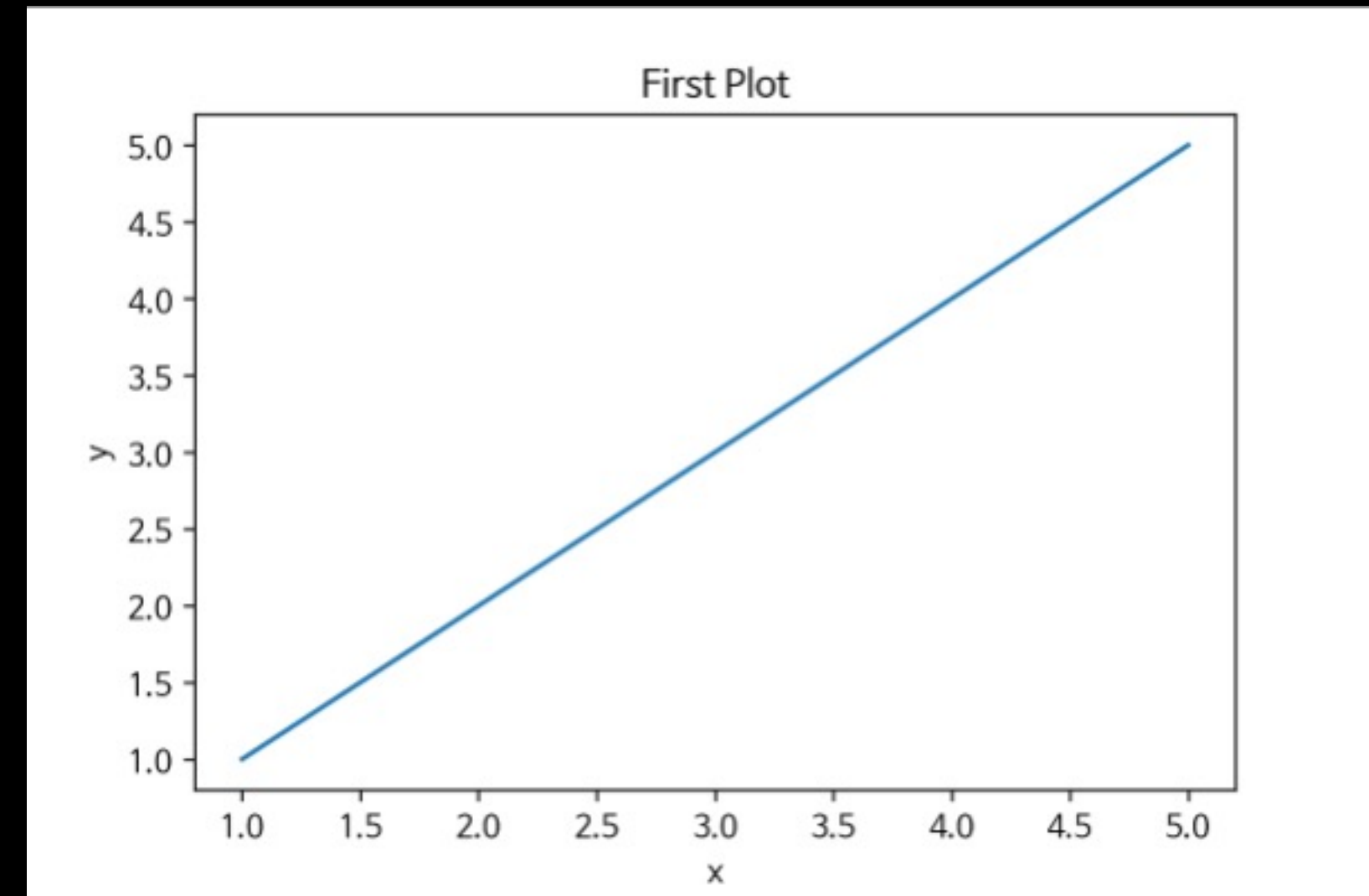
그래프 저장하기

머신러닝과 데이터 처리

데이터 탐색

시각화를 위한
Matplotlib

```
x = [1, 2, 3, 4, 5]
y = [1, 2, 3, 4, 5]
fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_title("First Plot")
ax.set_xlabel("x")
ax.set_ylabel("y")
fig.set_dpi(300)
fig.savefig("first_plot.png")
```





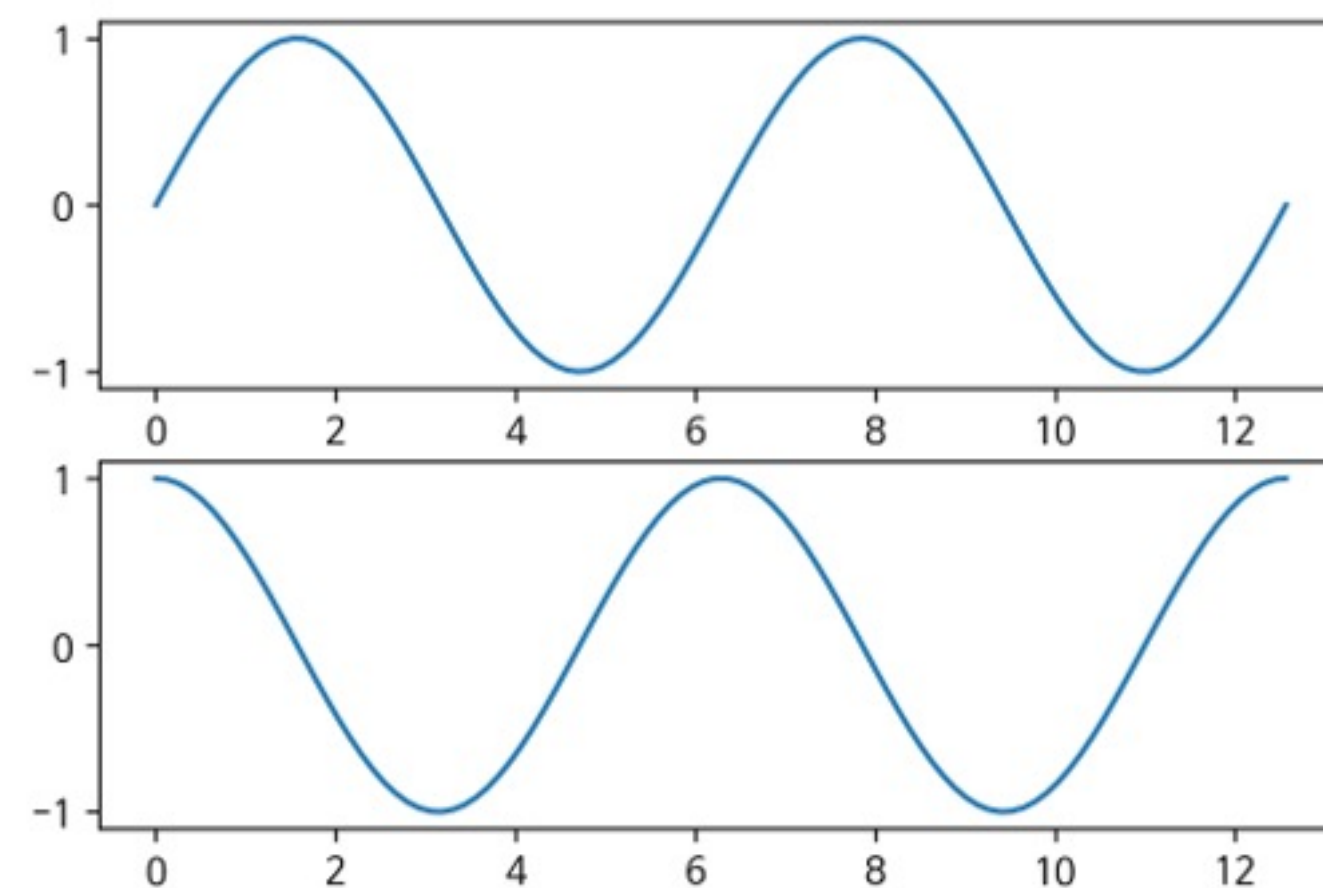
그래프 여러 개 그리기

머신러닝과 데이터 처리

데이터 탐색

시각화를 위한
Matplotlib

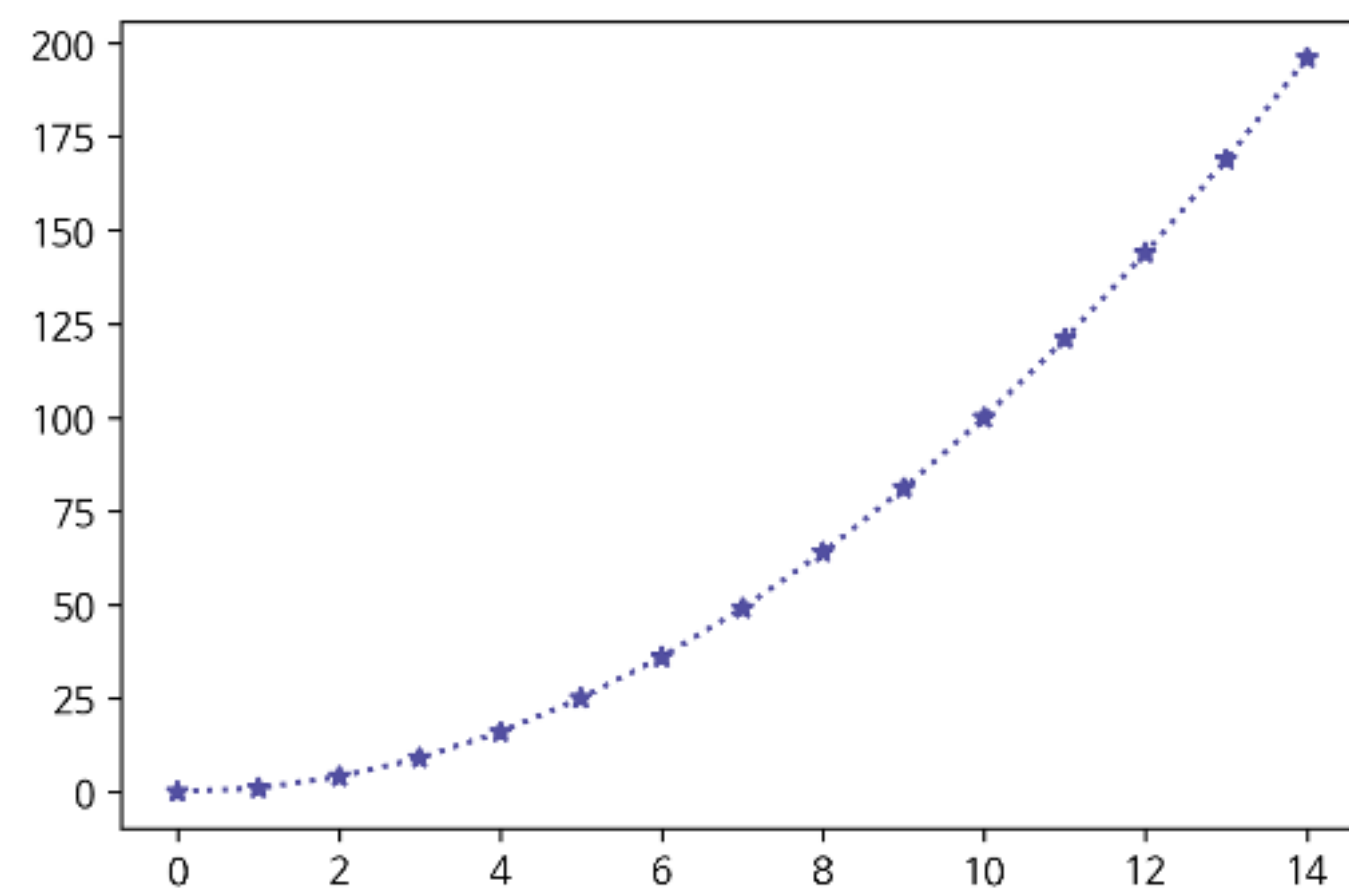
```
x = np.linspace(0, np.pi*4, 100)
fig, axes = plt.subplots(2, 1)
axes[0].plot(x, np.sin(x))
axes[1].plot(x, np.cos(x))
```





Line plot

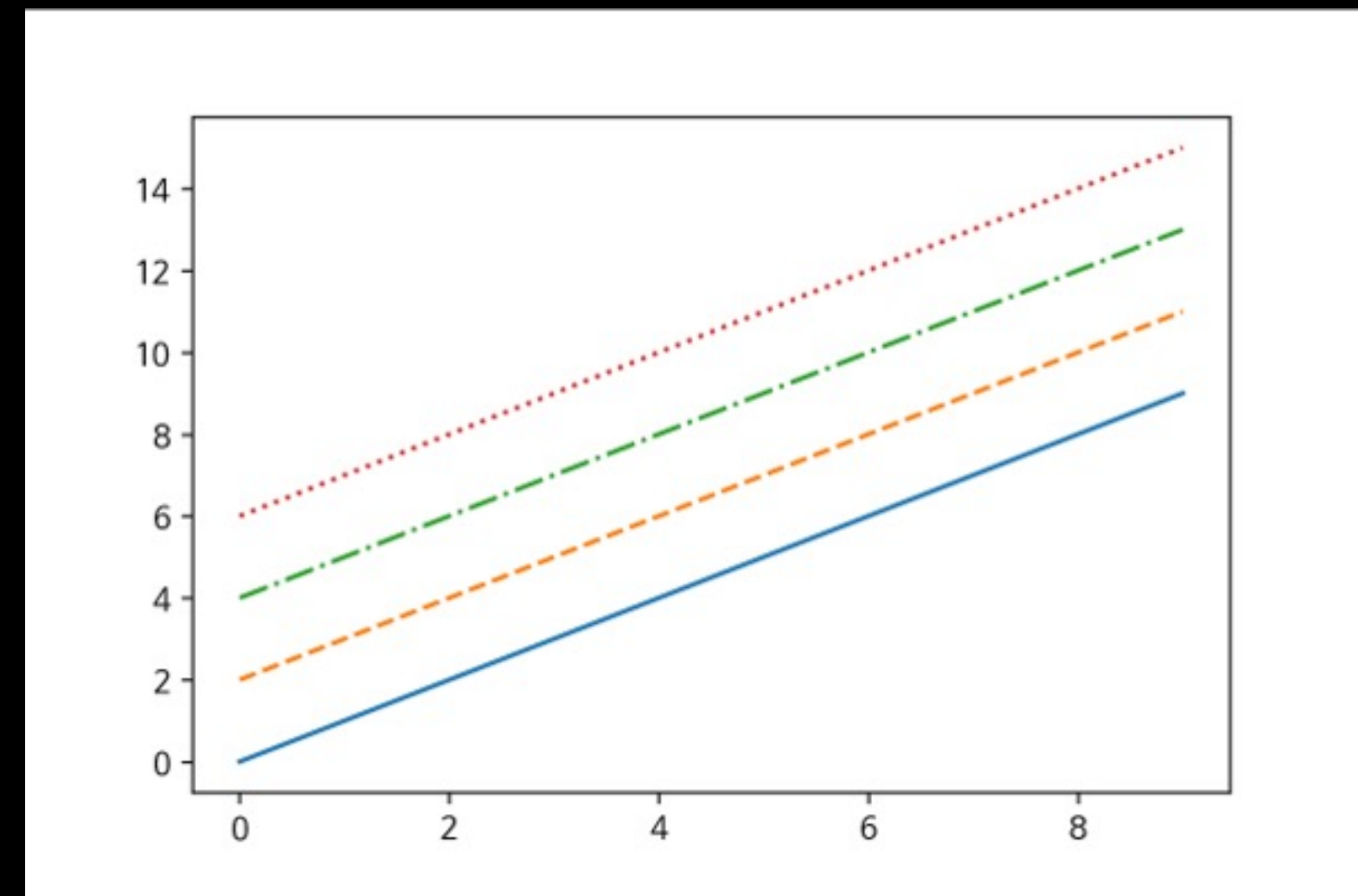
```
fig, ax = plt.subplots()
x = np.arange(15)
y = x ** 2
ax.plot(
    x, y,
    linestyle=":",
    marker="*",
    color="#524FA1"
)
```





Line style

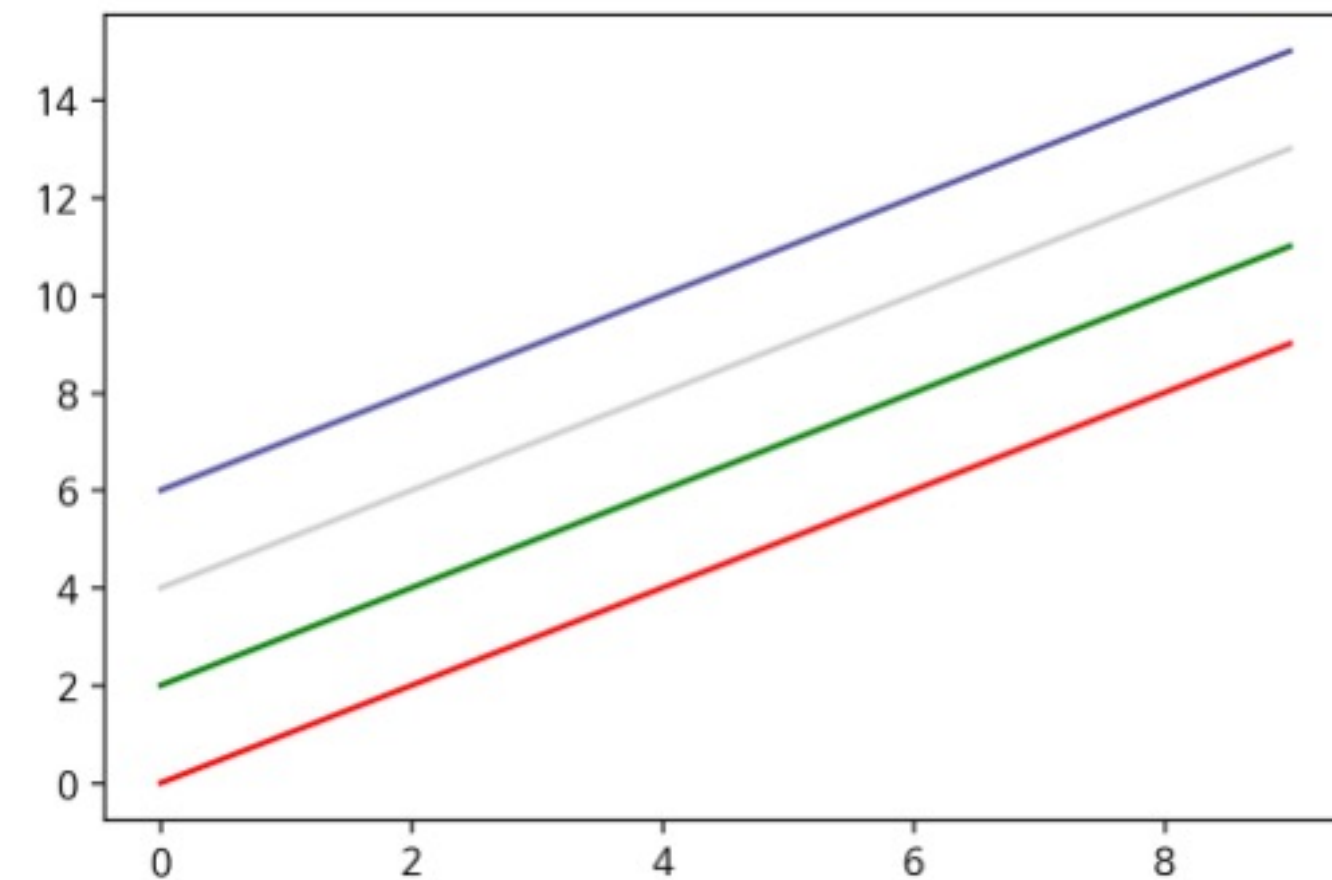
```
x = np.arange(10)
fig, ax = plt.subplots()
ax.plot(x, x, linestyle="-")
# solid
ax.plot(x, x+2, linestyle="--")
# dashed
ax.plot(x, x+4, linestyle="-.")
# dashdot
ax.plot(x, x+6, linestyle=":")
# dotted
```





Line color

```
x = np.arange(10)
fig, ax = plt.subplots()
ax.plot(x, x, color="r")
ax.plot(x, x+2, color="green")
ax.plot(x, x+4, color='0.8')
ax.plot(x, x+6, color="#524FA1")
```





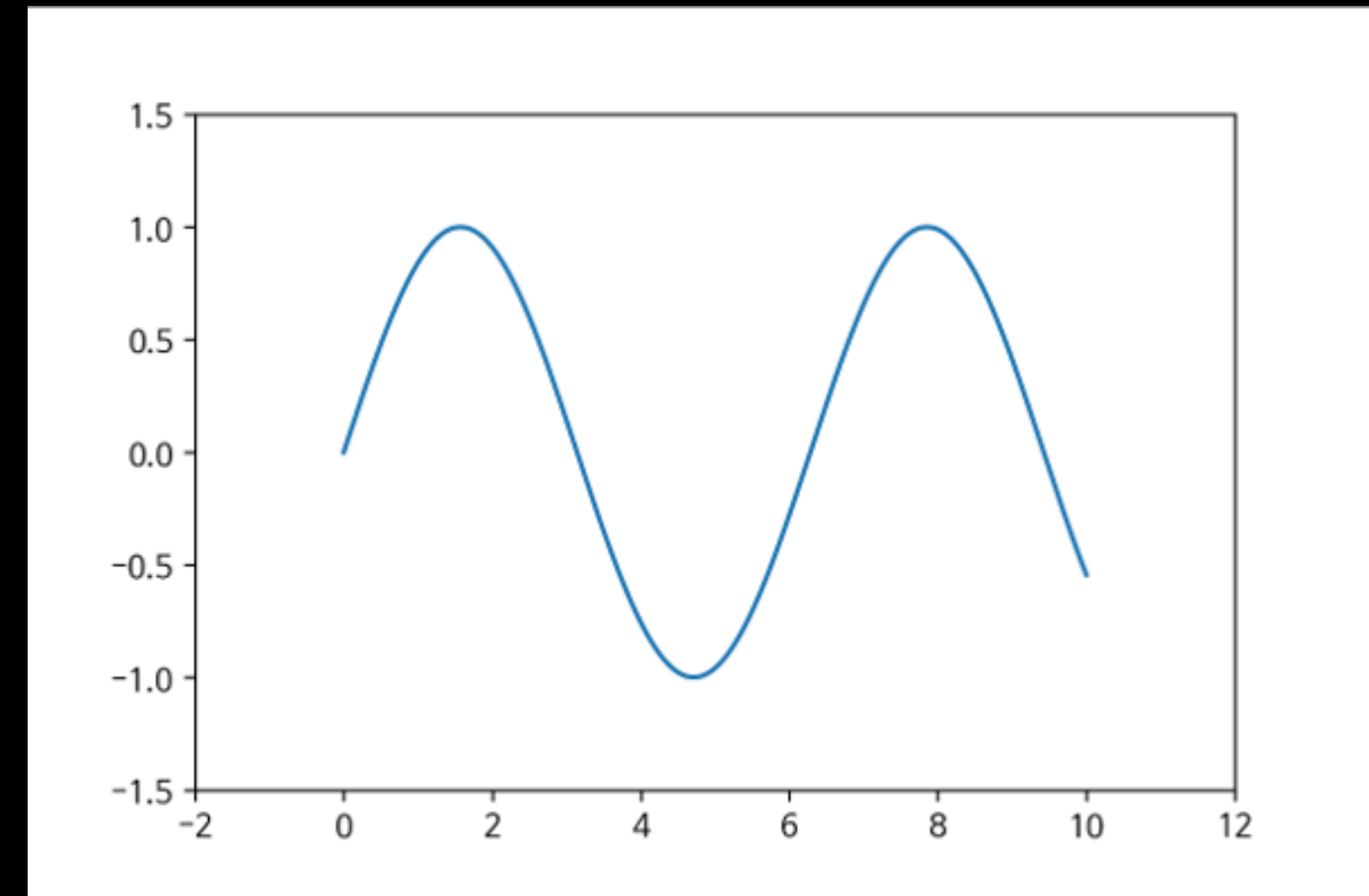
축 경계 조정하기

머신러닝과 데이터 처리

데이터 탐색

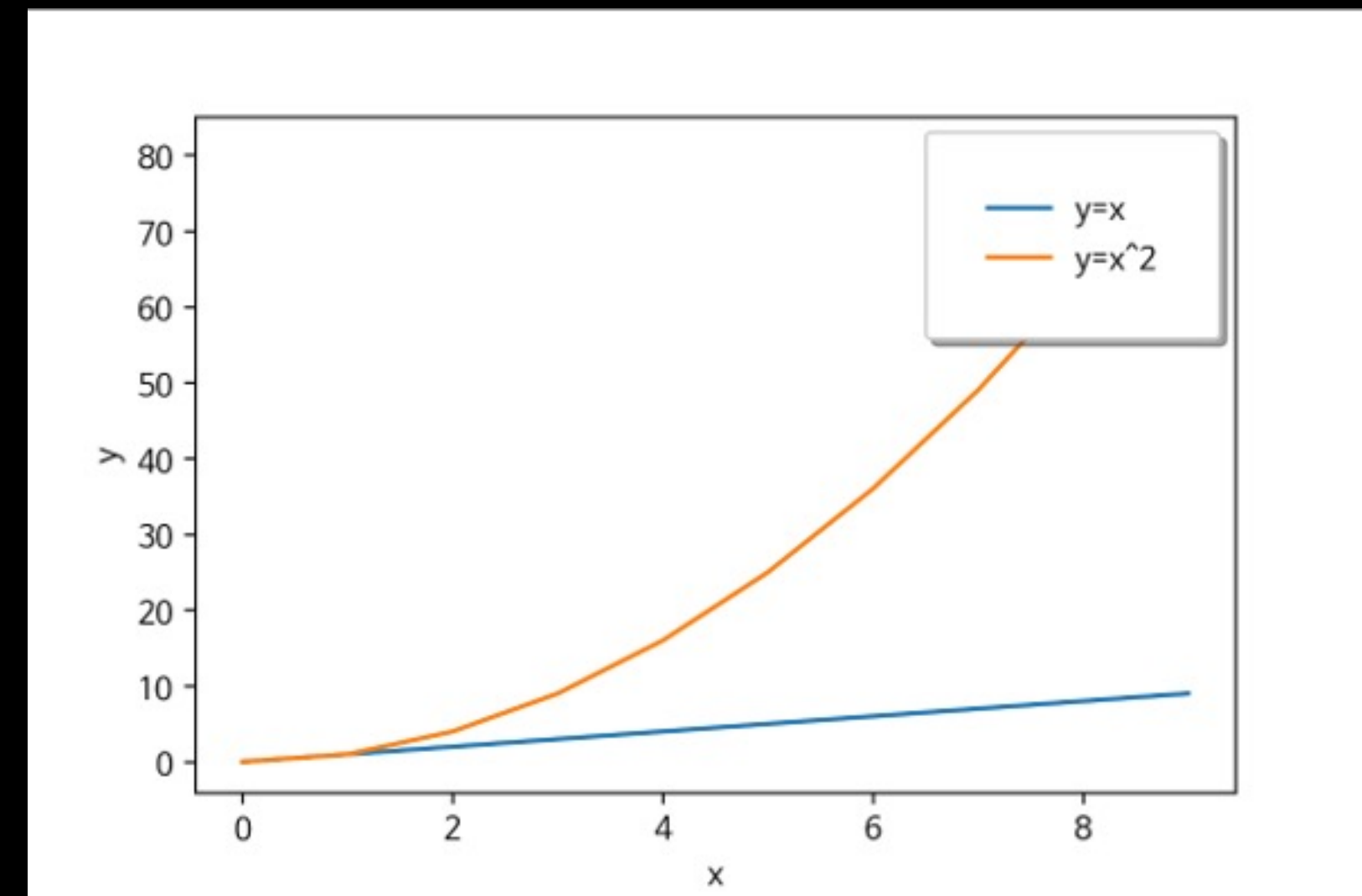
시각화를 위한
Matplotlib

```
x = np.linspace(0, 10, 1000)
fig, ax = plt.subplots()
ax.plot(x, np.sin(x))
ax.set_xlim(-2, 12)
ax.set_ylim(-1.5, 1.5)
```

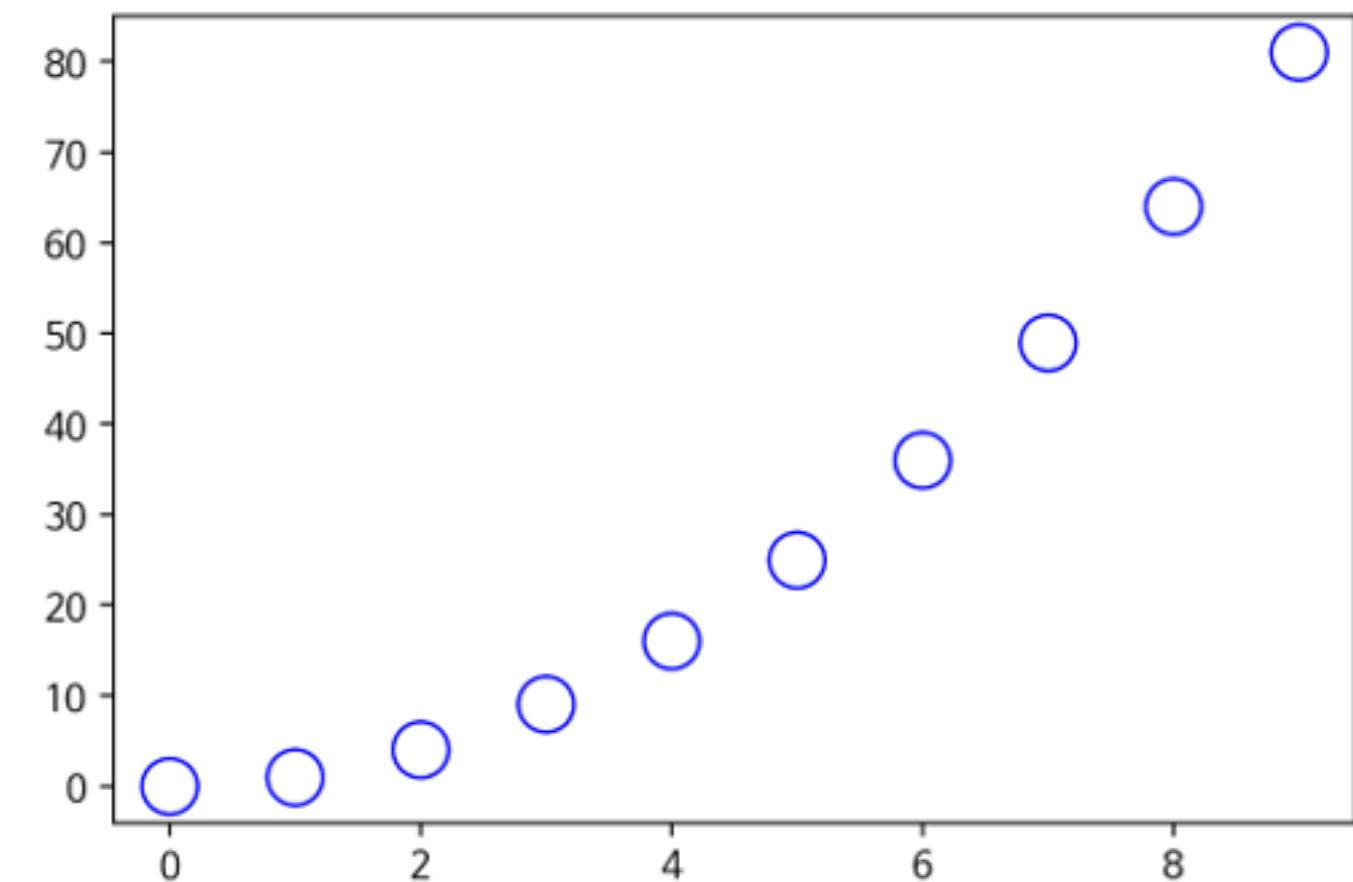




```
fig, ax = plt.subplots()
ax.plot(x, x, label='y=x')
ax.plot(x, x**2, label='y=x^2')
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.legend(
    loc='upper right',
    shadow=True,
    fancybox=True,
    borderpad=2
)
```



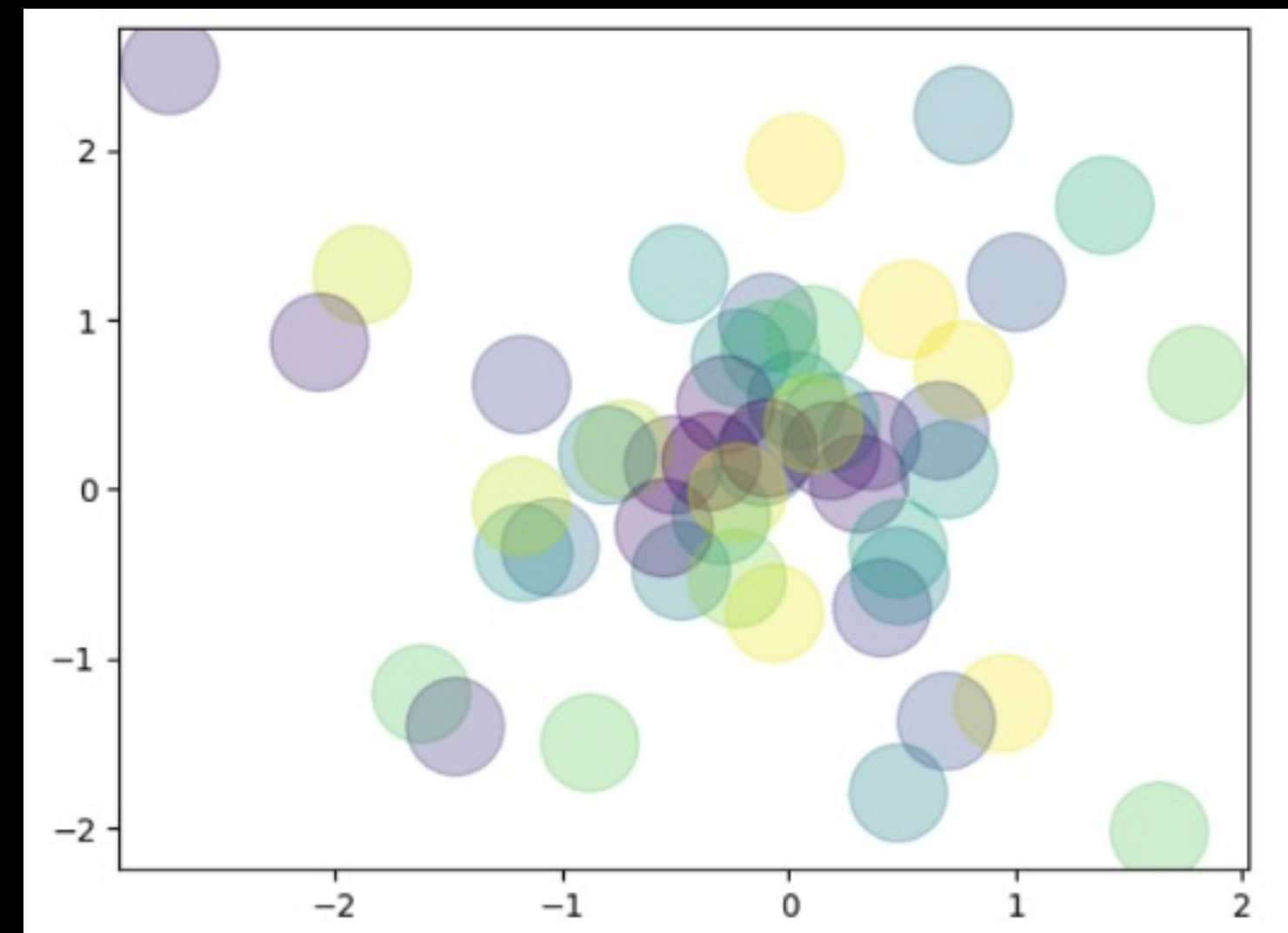
```
fig, ax = plt.subplots()
x = np.arange(10)
ax.plot(
    x, x**2, "o",
    markersize=15,
    markerfacecolor='white',
    markeredgecolor="blue"
)
```





Scatter

```
fig, ax = plt.subplots()
x = np.random.randn(50)
y = np.random.randn(50)
colors = np.random.randint(0, 100,
50)
sizes = 500 * np.pi *
np.random.rand(50) ** 2
ax.scatter(
    x, y, c=colors, s=sizes,
alpha=0.3
)
```





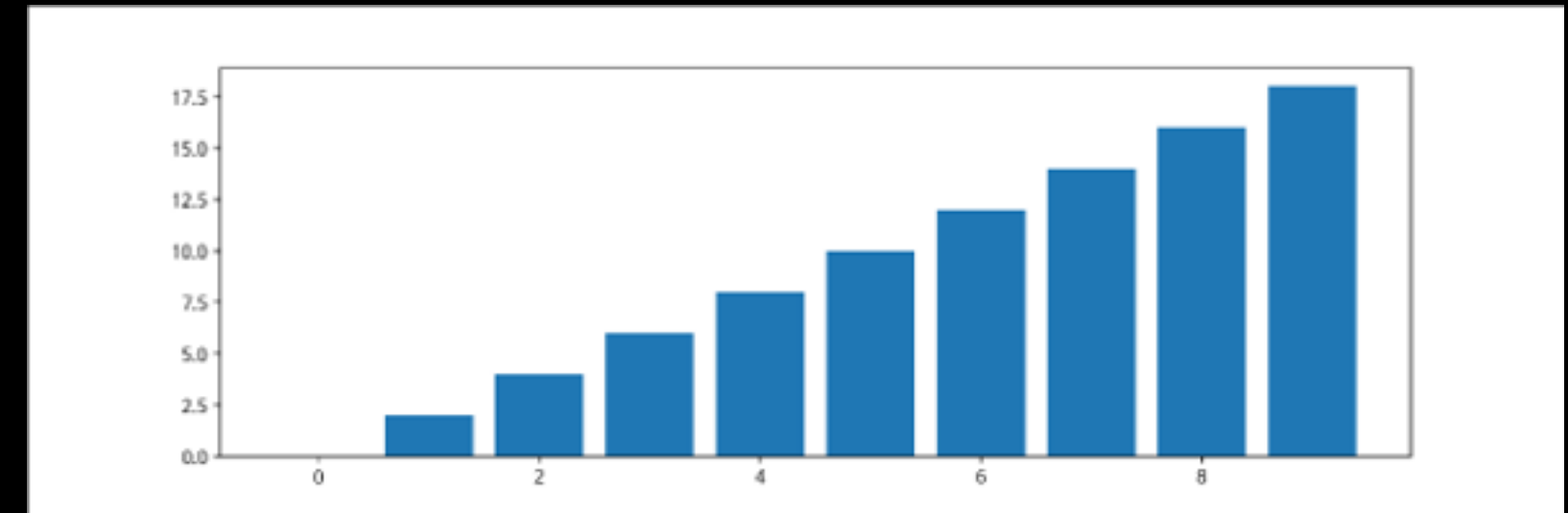
Bar chart

머신러닝과 데이터 처리

데이터 탐색

시각화를 위한
Matplotlib

```
# bar  
x = np.arange(10)  
fig, ax =  
plt.subplots(figsize=(12, 4))  
ax.bar(x, x*2)
```

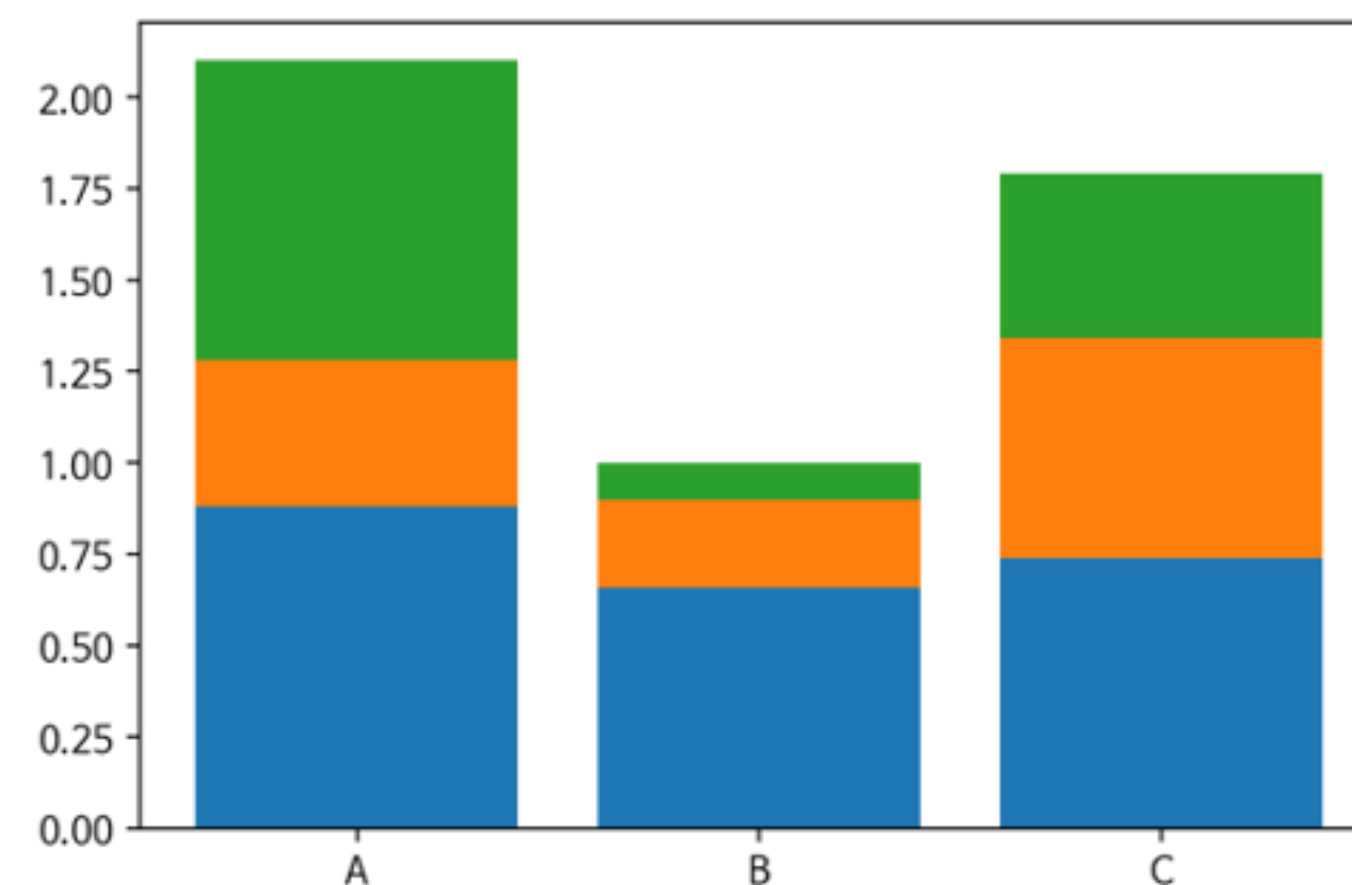




Bar chart

```
x = np.random.rand(3)
y = np.random.rand(3)
z = np.random.rand(3)
data = [x, y, z]

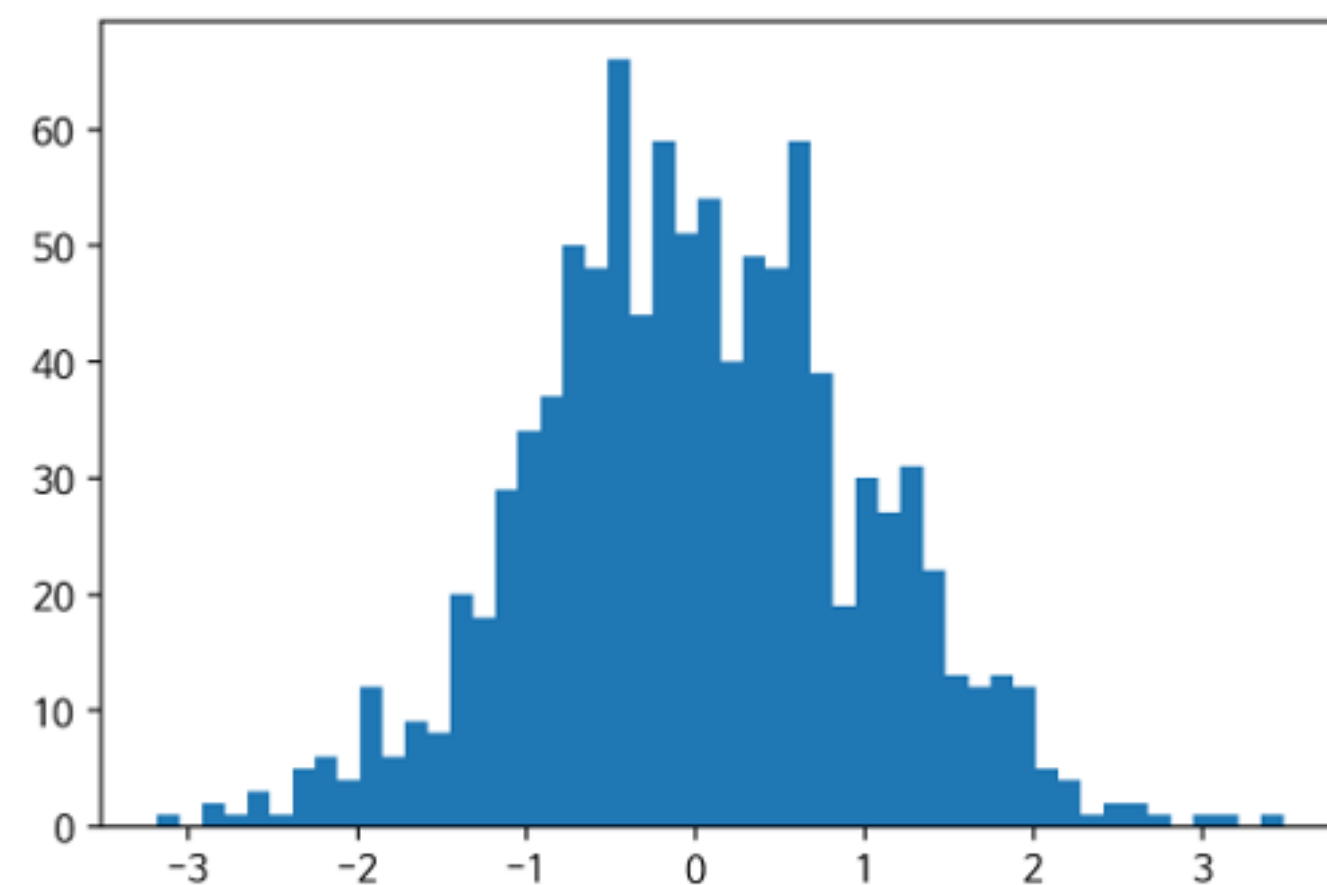
fig, ax = plt.subplots()
x_ax = np.arange(3)
for i in x_ax:
    ax.bar(x_ax, data[i],
           bottom=np.sum(data[:i], axis=0))
ax.set_xticks(x_ax)
ax.set_xticklabels(["A", "B", "C"])
```





Histogram

```
fig, ax = plt.subplots()
data = np.random.randn(1000)
ax.hist(data, bins=50)
```





- Matplotlib 기반의 파이썬 라이브러리

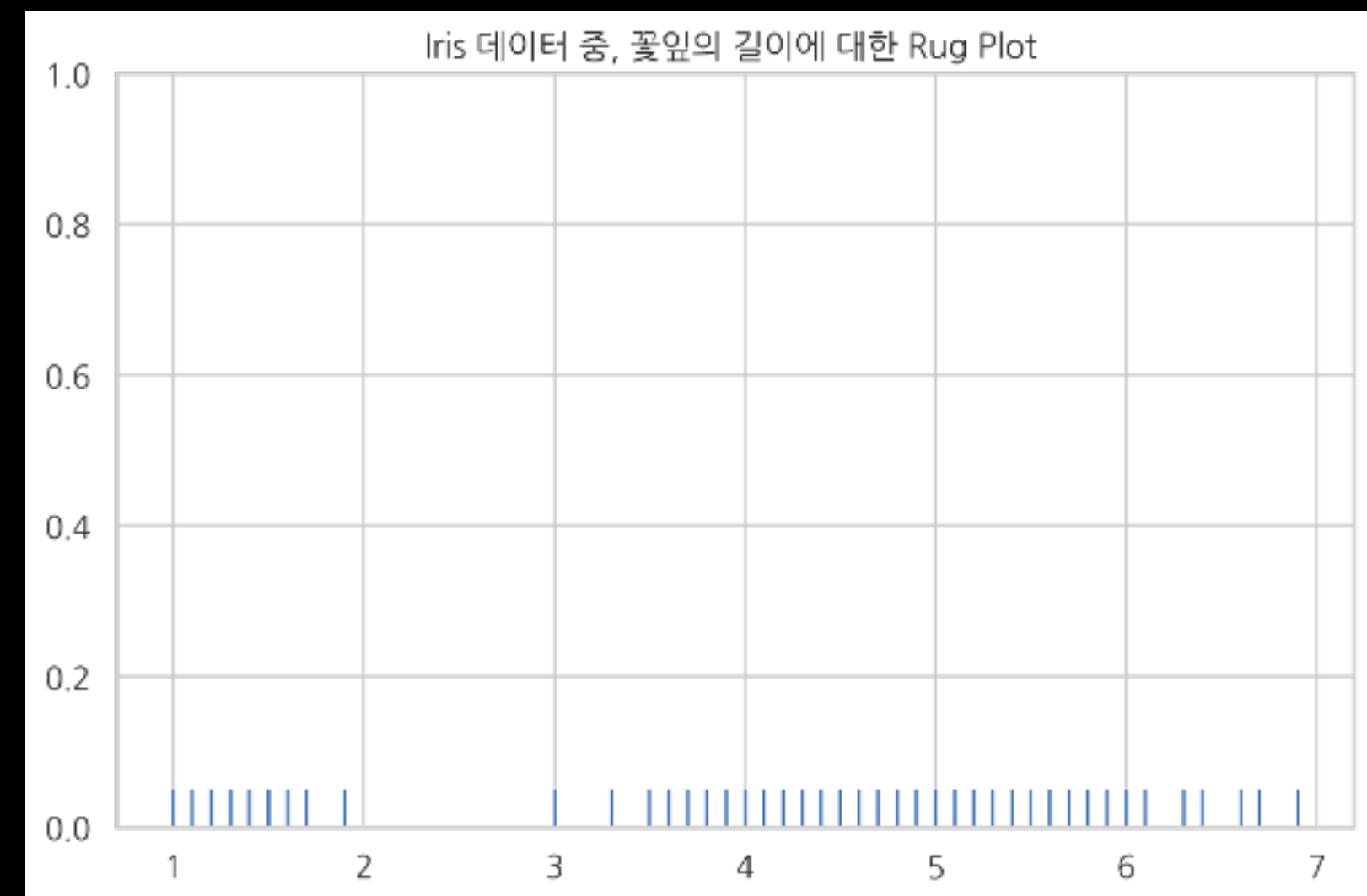


- Statsmodels의 통계 기능 활용

- Matplotlib에서 제공하지 않는 통계용 차트 기능 제공

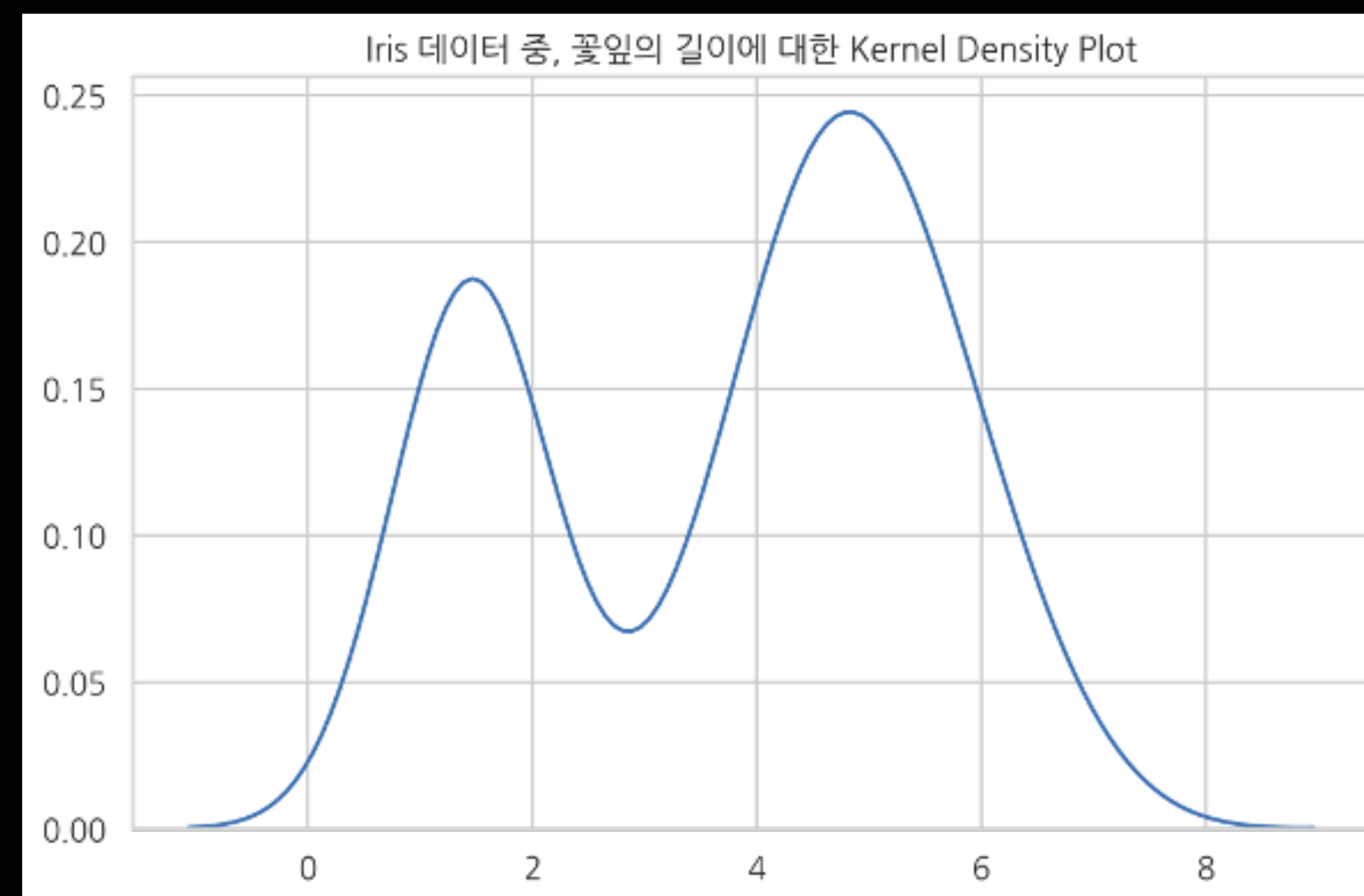

```
x = iris.petal_length.values

sns.rugplot(x)
plt.title("Iris 데이터 중, 꽃잎의 길이에 대한 Ru
g Plot ")
plt.show()
```



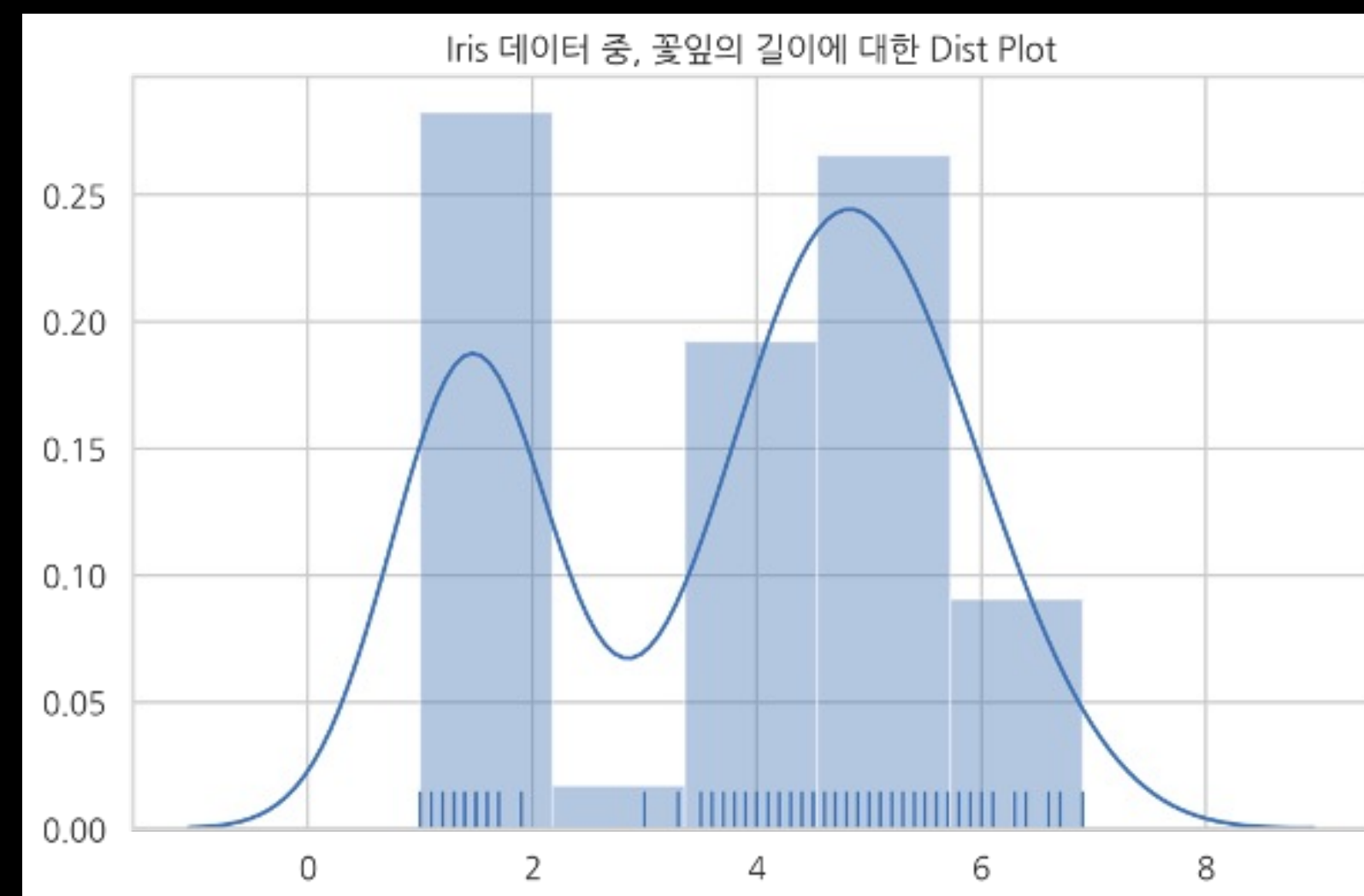


```
sns.kdeplot(x)  
plt.title("Iris 데이터 중, 꽃잎의 길이에 대한 Kernel Density Plot")  
plt.show()
```

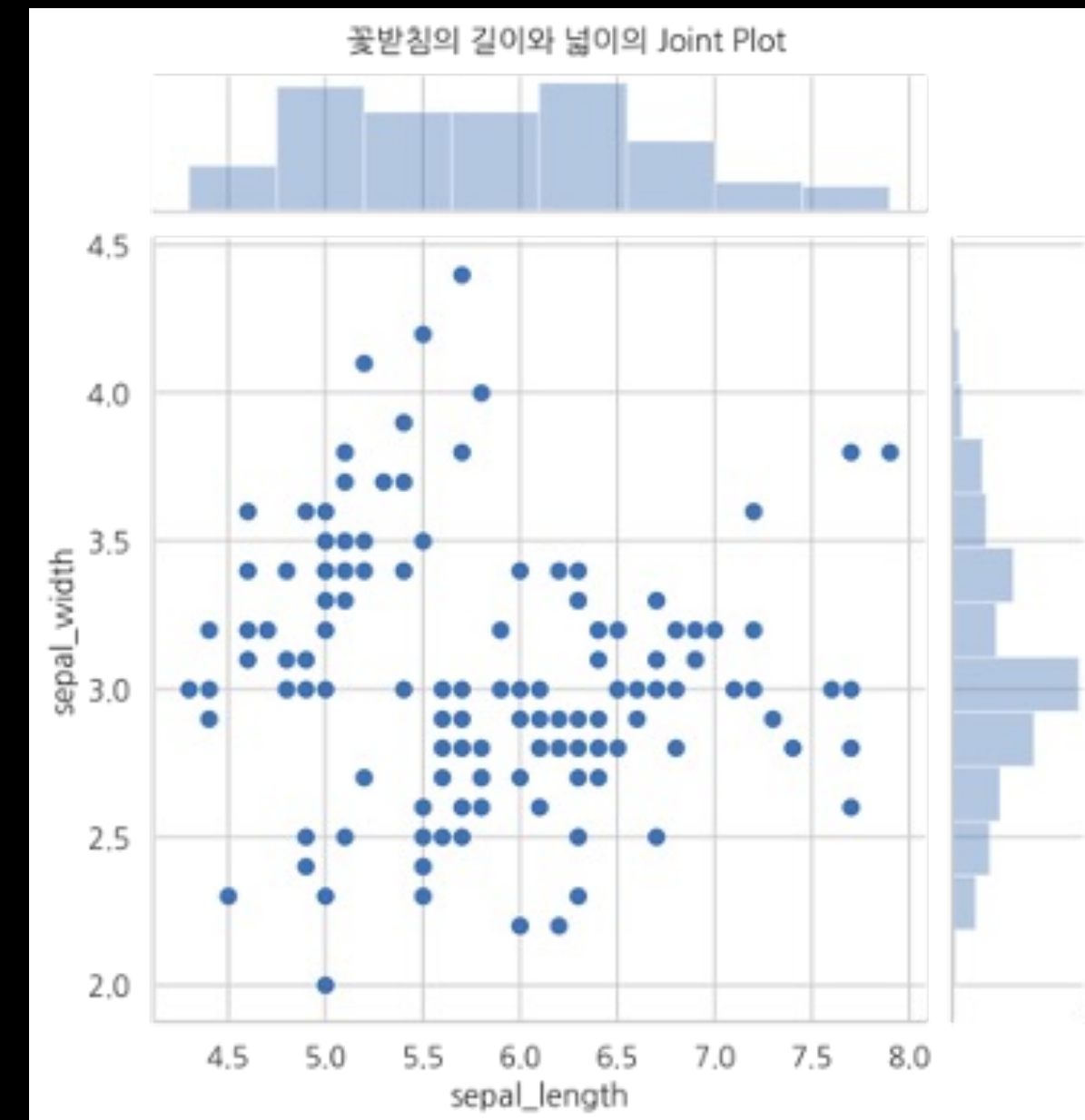




```
sns.distplot(x, kde=True, rug=True)  
plt.title("Iris 데이터 중, 꽃잎의 길이에 대한 Dis  
t Plot")  
plt.show()
```



```
sns.jointplot(x="sepal_length", y="sepal_width",  
data=iris)  
plt.suptitle("꽃받침의 길이와 넓이의 Joint Plot", y=1.02)  
plt.show()
```





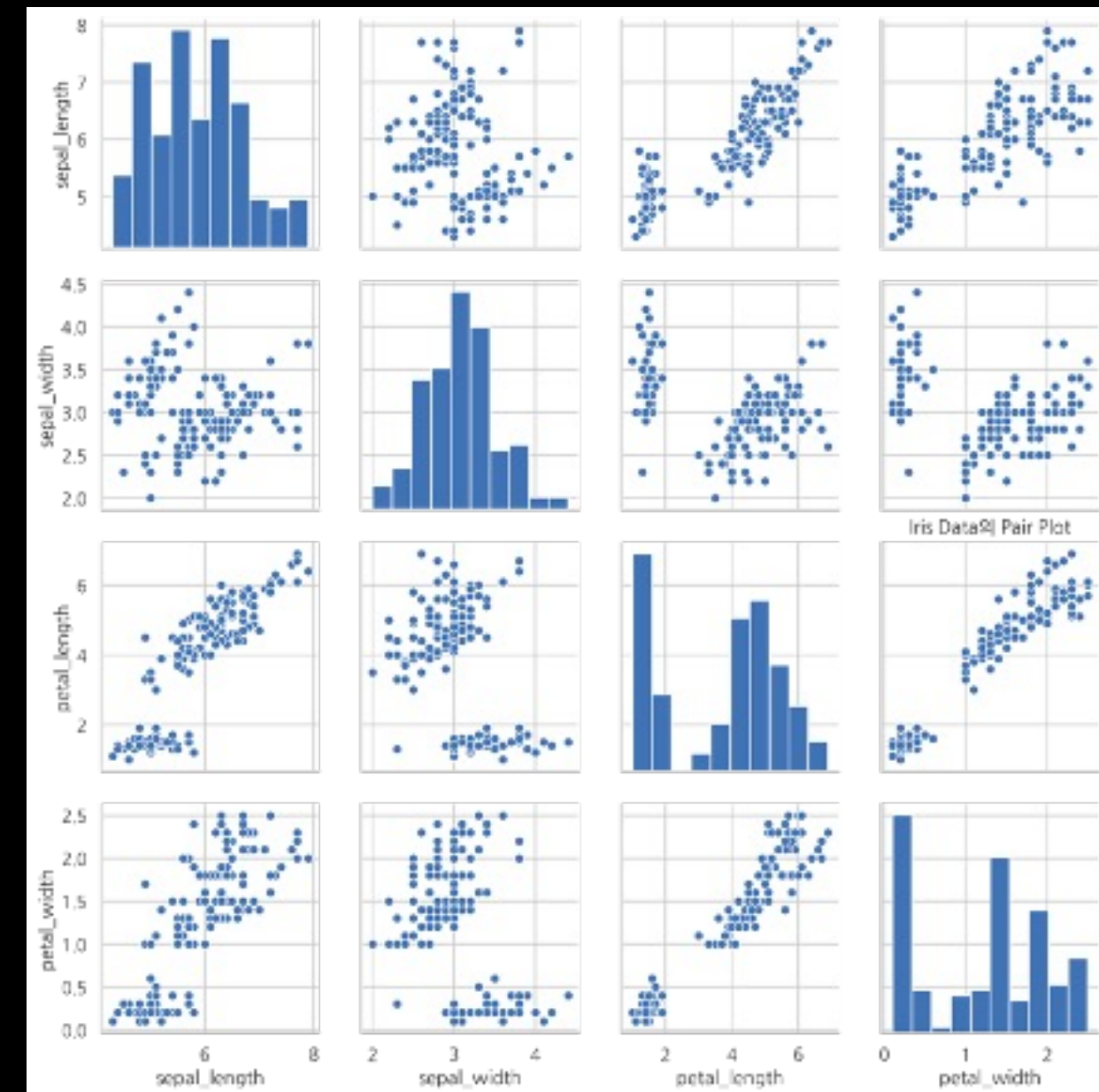
pairplot

머신러닝과 데이터 처리

데이터 탐색

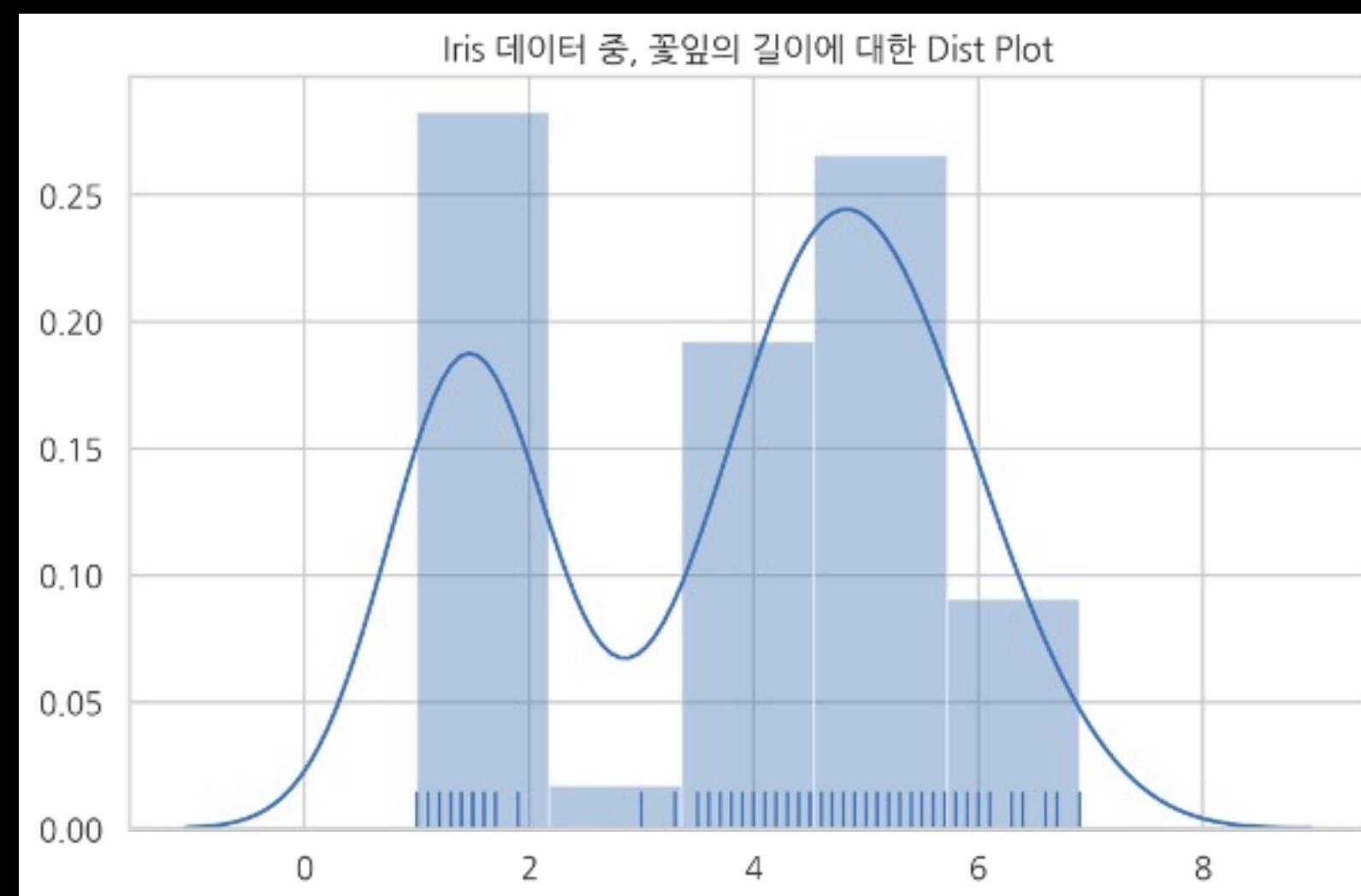
시각화를 위한
Matplotlib

```
sns.pairplot(iris)
plt.title("Iris Data의 Pair Plot")
plt.show()
```





```
sns.distplot(x, kde=True, rug=True)  
plt.title("Iris 데이터 중, 꽃잎의 길이에 대한 Dis  
t Plot")  
plt.show()
```





지금까지

머신러닝과 데이터 처리

데이터 탐색

시각화를 위한
Matplotlib

	PassengerId	Survived	Pclass
0	1	0	3
1	2	1	1
2	3	1	3

데이터



데이터 조작 도구



시각화 도구



지금까지

머신러닝과 데이터 처리

데이터 탐색

시각화를 위한
Matplotlib

